

RAAS PAL

Robot Recommendation System

Architecture & Design Specification

Version 1.0 | May 2026
Stack: Java / Spring Boot • Next.js • PostgreSQL • Claude AI

This document covers system architecture, role design, entity design,
API endpoints, security model, and package structure.

1. System Overview

Raas Pal's Robot Recommendation System is a web-based platform that enables the Raas Pal team to analyze customer requirements (industry, environment, payload, budget, connectivity) against a structured robot catalog, then generate AI-powered recommendations with supporting reports.

1.1 Technology Stack

Layer	Technology	Purpose
Backend	Java 21 + Spring Boot 3.x	REST API, business logic, security
Frontend	Next.js 14 (React)	Requirement wizard, dashboards, reports
Database	PostgreSQL 16	Primary data store
Cache	Redis	Sessions, AI response caching
AI	Claude API via Spring AI	Recommendation reasoning, NL explanations
Embeddings	OpenAI / pgvector	Semantic similarity search
Auth	Spring Security + JWT	Role-based access control
Export	JasperReports / iText	PDF recommendation reports

1.2 Development Milestones

Recommended solo-developer build order:

Phase	Scope	Outcome
1	Database schema + seed data	PostgreSQL tables, 15-20 robots seeded
2	Spring Boot skeleton	Entities, CRUD, OpenAPI docs
3	Recommendation engine	Filter rules, weighted scoring, ranking
4	AI integration	Spring AI + Claude explanations + pgvector
5	Next.js frontend	Wizard, results, comparison view
6	Admin panel	Robot catalog CRUD, scoring config
7	Reports + export	PDF generation
8	Deployment	Docker Compose → cloud (Render / Railway)

2. Role Design

2.1 Role Enum

Three roles are defined system-wide:

Role	Description	Dashboard Route
ADMIN	Full system access — users, robots, all customers, audit logs, scoring config	/dashboard/admin
RAASPAL_TEAM	Customer management, requirement intake, generate recommendations and reports — no user mgmt or audit log	/dashboard/team
CUSTOMER	Own requirements, recommendations, reports, file uploads only — data isolation enforced server-side	/dashboard/customer

2.2 Dashboard Access Logic

After a successful login the backend issues a JWT. The frontend reads the role claim and redirects automatically:

JWT Redirect Flow		
ADMIN	→ /dashboard/admin	(full system)
RAASPAL_TEAM	→ /dashboard/team	(operations)
CUSTOMER	→ /dashboard/customer	(own data only)
Backend enforces ownership — role alone is not enough for CUSTOMER. Every CUSTOMER request is verified: requirement.customer_profile_id == currentUser.customerProfileId		

2.3 JWT Claims Structure

JWT Payload	
{	
"sub":	"user@email.com",
"userId":	"uuid",
"role":	"CUSTOMER",
"name":	"Jane Doe",
"customerProfileId":	"uuid", // only present for CUSTOMER role
"iat":	1234567890,

```
"exp": 1234567890
}
```

The `customerProfileId` claim embedded in the token lets the backend enforce data isolation for customers without an extra database lookup on every request.

3. Entity Design

3.1 User

All three roles share a single User table and the same `/api/v1/auth/login` endpoint. ADMIN and RAASPAL_TEAM are internal users with no profile extension. CUSTOMER users must have a corresponding CustomerProfile row.

Column	Type	Constraints
id	UUID	PK, auto-generated
email	VARCHAR	Unique, not null
password	VARCHAR	bcrypt hashed, not null
full_name	VARCHAR	Not null
role	ENUM(ADMIN, RAASPAL_TEAM, CUSTOMER)	Not null
is_active	BOOLEAN	Default true
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.2 CustomerProfile

Created atomically with the User when RAASPAL_TEAM onboards a customer. One profile per customer user (1:1 with User).

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → User (unique)
company_name	VARCHAR	Not null
industry	VARCHAR	
contact_phone	VARCHAR	
address	TEXT	
notes	TEXT	Internal notes by RAASPAL_TEAM
created_by	UUID	FK → User (the team member who onboarded)
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.3 Requirement

The central intake entity. Ownership is anchored to CustomerProfile. All role-based ownership checks use: requirement.customer_profile_id == currentUser.customerProfileId.

Column	Type	Notes
id	UUID	PK
customer_profile_id	UUID	FK → CustomerProfile (ownership anchor)
title	VARCHAR	
description	TEXT	Free-text customer needs
min_payload_kg	DECIMAL	
max_payload_kg	DECIMAL	
min_reach_mm	INT	
max_reach_mm	INT	
environment	ENUM(INDOOR, OUTDOOR, CLEANROOM, HAZARDOUS)	
budget_usd	DECIMAL	
priority_notes	TEXT	Additional context
status	ENUM(DRAFT, SUBMITTED, IN_PROGRESS, COMPLETED)	
created_by	UUID	FK → User
created_at	TIMESTAMP	Auto
updated_at	TIMESTAMP	Auto

3.4 Other Entities (unchanged)

The following entities require no structural change from the base design. Ownership of recommendations and reports is derived transitively through the requirement's customer_profile_id:

Entity	Purpose
Robot	Robot catalog — model, manufacturer, specs summary
RobotSpec	Detailed technical specifications per robot
Recommendation	AI-generated recommendation session per requirement
RecommendationItem	Individual robot ranked within a recommendation
Score	Per-criterion score breakdown for each item
FileUpload	Uploaded documents linked to requirements or reports
Report	Generated PDF report record
AuditLog	Immutable system event log (ADMIN access only)

4. Backend Package Structure

Spring Boot project root: com.raaspal.robotrecommendation

Package Layout

```
com.raaspal.robotrecommendation
|
├─ auth/
|   ├── controller/    AuthController
|   ├── service/       AuthService, TokenService
|   ├── dto/           LoginRequest, LoginResponse, TokenRefreshRequest
|   └─ security/       JwtFilter, JwtUtil, SecurityConfig,
|                       UserDetailsServiceImpl, JwtClaims
|
├─ user/
|   ├── controller/    UserController
|   ├── service/       UserService
|   ├── repository/    UserRepository
|   ├── entity/        User
|   └─ dto/            UserResponse, CreateUserRequest,
|                       UpdateUserRequest, ChangePasswordRequest
|
├─ customer/
|   ├── controller/    CustomerController
|   ├── service/       CustomerService (creates User + CustomerProfile
atomically)
|   ├── repository/    CustomerProfileRepository
|   ├── entity/        CustomerProfile
|   └─ dto/            CreateCustomerRequest, CustomerProfileResponse,
|                       UpdateCustomerProfileRequest
|
├─ robot/              RobotController, RobotService, Robot, RobotSpec
├─ requirement/        RequirementController, RequirementService,
Requirement
├─ recommendation/    RecommendationController, Recommendation,
RecommendationItem
├─ scoring/            ScoringService, ScoreEngine, Score
├─ ai/                 AiRecommendationService, EmbeddingService,
PromptBuilder
```

```

├─ file/           FileController, FileService, FileUpload
├─ report/         ReportController, ReportService, Report
├─ audit/          AuditLogController, AuditLogService, AuditLog
├─
├─ common/
│   ├── exception/  GlobalExceptionHandler, ResourceNotFoundException
│   ├── response/   ApiResponse<T>, PagedResponse<T>
│   ├── enums/      Role, RequirementStatus, RecommendationStatus,
Environment
│   ├── audit/      AuditAspect (AOP listener)
│   └─ security/    CurrentUser annotation, SecurityUtil
                        (helper to extract userId / customerProfileId from
JWT)

```

5. Security Configuration

5.1 Access Rules Summary

Tier	Endpoints	Rule
Public	POST /auth/login, POST /auth/refresh	No token required
Authenticated	POST /auth/logout, GET+POST /files/**	Any valid JWT
ADMIN only	GET/DELETE /users/**, /audit-logs/**, DELETE /robots/{id}, DELETE /recommendations/{id}	ADMIN role
ADMIN + TEAM	POST/PUT /robots/**, POST/GET /customers/**, POST /recommendations, POST /reports	ADMIN or RAASPAL_TEAM
All roles (owned)	CRUD /requirements/**, GET /recommendations/**, GET /reports/**	Role + ownership check in service layer

5.2 Customer Onboarding Flow

Customers are created by RAASPAL_TEAM — they do not self-register. The server creates both User and CustomerProfile atomically in one transaction:

Onboarding API Call — POST /api/v1/customers

Request body (sent by RAASPAL_TEAM):

```
{
  "email": "client@company.com",
  "fullName": "Jane Doe",
  "temporaryPassword": "...",
  "companyName": "Acme Corp",
  "industry": "Automotive",
  "contactPhone": "...",
  "address": "..."
}
```

Server atomically:

- 1. Creates User (role = CUSTOMER, is_active = true)
- 2. Creates CustomerProfile linked to that User
- 3. Returns both User and CustomerProfile in response

Customer Login — POST /api/v1/auth/login

Request: { email, password }

Response: JWT containing role=CUSTOMER, customerProfileId=<uuid>

Frontend: redirects to /dashboard/customer

6. API Endpoint Permissions

Base path: /api/v1 | Format: JSON | Auth: Bearer JWT

6.1 Auth — /api/v1/auth

Method	Path	Access
POST	/login	Public — all roles
POST	/refresh	Public
POST	/logout	Authenticated

6.2 Users — /api/v1/users

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Create any user	—	—
GET	/	All users	—	—
GET	/id	Any	—	Self only
PUT	/id	Any	—	Self only
PUT	/id/password	Any	Self	Self
PUT	/id/role	Yes	—	—
DELETE	/id	Yes	—	—

6.3 Customers — /api/v1/customers

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Create customer	Create customer	—
GET	/	All profiles	All profiles	—
GET	/id	Yes	Yes	Own profile
PUT	/id	Yes	Yes	Own profile
DELETE	/id	Yes	—	—
GET	/id/requirements	Yes	Yes	Own only

6.4 Robot Catalog — /api/v1/robots

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST/PUT	/, /id, /specs, /image, /datasheet	Yes	Yes	—

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
DELETE	/id	Yes	—	—
GET	/, /id, /search	Yes	Yes	Yes (view)

6.5 Requirements — /api/v1/requirements

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Yes (any customer)	Yes (any customer)	Own profile only
GET	/	All	All	Own only (auto-filtered)
GET	/id	Yes	Yes	Own only
PUT	/id	Yes	Yes	Own, DRAFT only
DELETE	/id	Yes	Yes	Own, DRAFT only
POST	/id/submit	Yes	Yes	Own only

6.6 Recommendations — /api/v1/recommendations

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Yes	Yes	— (view only)
GET	/	All	All	Own only (auto-filtered)
GET	/id	Yes	Yes	Own only
GET	/id/scores	Yes	Yes	Own only
DELETE	/id	Yes	—	—

6.7 Reports — /api/v1/reports

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/	Yes	Yes	—
GET	/	All	All	Own only
GET	/id/download	Yes	Yes	Own only

6.8 Files — /api/v1/files

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
POST	/upload	Yes	Yes	Yes (own entity)
GET	/id/download	Yes	Yes	Own file only

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
DELETE	/id}	Yes	Yes	Own file, if not locked

6.9 Audit Logs — /api/v1/audit-logs

Method	Path	ADMIN	RAASPAL_TEAM	CUSTOMER
GET	/	Yes	—	—
GET	/entityType}/{entityId}	Yes	—	—

7. Key Design Decisions

7.1 Two-phase recommendation engine

Hard filtering eliminates robots that cannot physically work (wrong payload, incompatible environment, over budget). AI scoring then ranks the remaining candidates. This keeps AI costs low — you embed and rank only qualified robots — and keeps the system auditable.

7.2 Modular monolith (not microservices)

For a solo developer at this stage, a modular monolith with clean package boundaries is correct. Each package encodes the eventual microservice split if needed, but the overhead of service-to-service communication and distributed tracing is avoided.

7.3 pgvector instead of a dedicated vector database

pgvector runs inside PostgreSQL, keeping the stack simple. It handles the volume of robot embeddings required at any realistic Raas Pal scale. A separate Pinecone or Weaviate instance adds operational complexity with no benefit at this stage.

7.4 Spring AI as the orchestrator

Spring AI provides a vendor-neutral abstraction over Claude and OpenAI. Provider switching or mixing requires no service logic changes.

7.5 Customers never self-register

RAASPAL_TEAM onboards customers. This ensures data quality, correct industry/company mapping, and prevents unauthorized access. The atomic creation of User + CustomerProfile in one transaction ensures the system is never in an inconsistent state.

7.6 customerProfileId in JWT

Embedding the customerProfileId in the JWT token eliminates one database lookup per request for ownership checks. The alternative (looking up CustomerProfile by userId on every call) adds latency and load. The tradeoff is that a role change requires token refresh, which is standard practice.

8. Recommended Next Steps

- **Step 1 — PostgreSQL schema:** Create all tables as defined in Section 3. Add indexes on foreign keys and the environment/status enum columns. Seed 15-20 robots.
- **Step 2 — Spring Boot skeleton:** Initialize project with Spring Web, Spring Data JPA, Spring Security, Spring AI. Create all entity classes and repositories. Expose `/api/v1/robots` as a test endpoint.
- **Step 3 — Auth module:** Implement JWT generation, JwtFilter, SecurityConfig, and the three-role access rules from Section 5.
- **Step 4 — Customer onboarding:** Implement CustomerService with the atomic User + CustomerProfile creation transaction.
- **Step 5 — Requirement intake:** Build the Requirement CRUD with ownership enforcement and status transitions.
- **Step 6 — Scoring engine:** Implement the hard filter rules and weighted multi-criteria scorer. Test with Postman before adding AI.
- **Step 7 — AI integration:** Add Spring AI + Claude API for recommendation explanation. Add pgvector for semantic similarity. Cache AI responses in Redis.
- **Step 8 — Next.js frontend:** Build the requirement wizard, results page, and comparison view against the stable API contract.

— End of Document —